

Artificial Intelligence Agents

Learning to Reason and Act in Multimodal and Embodied Environments

Alessandro Suglia



Outline

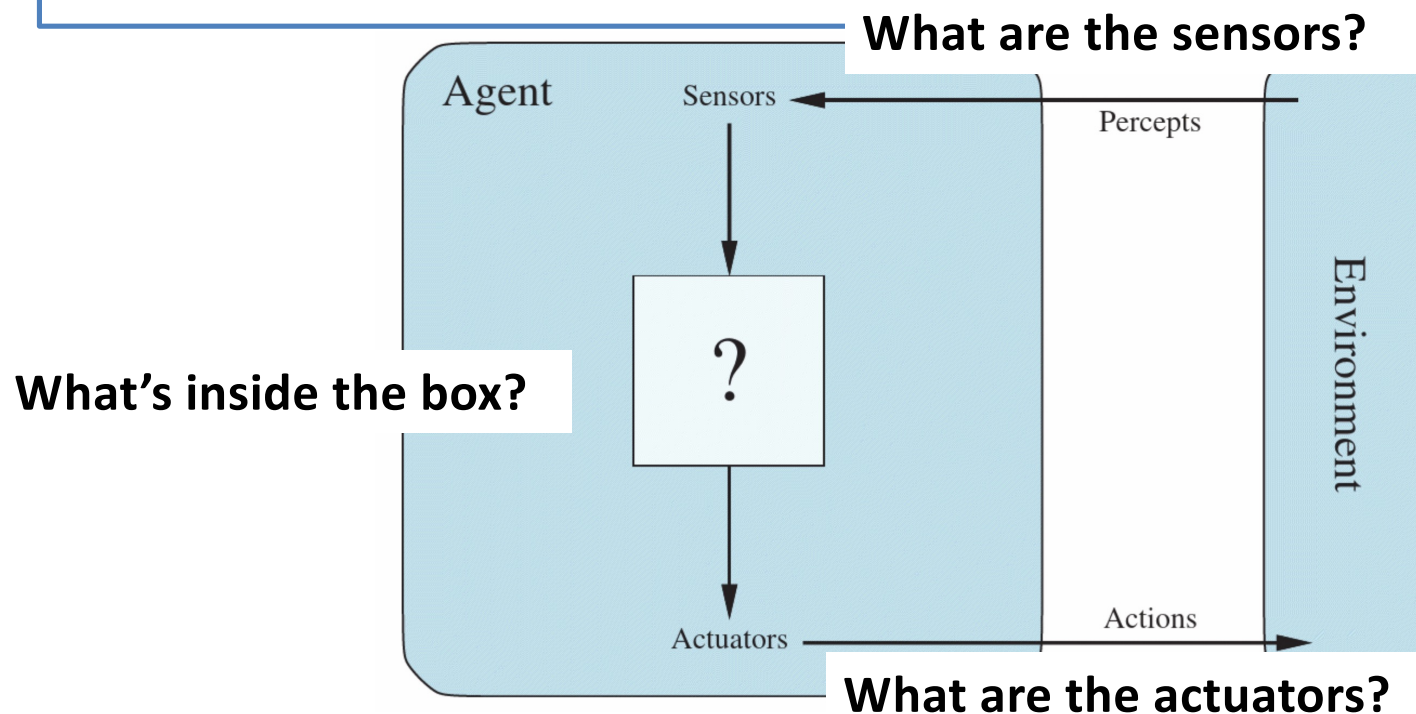
- Aim: understand how to implement AI agentic solutions for a variety of different multimodal and embodied environments
- Outline:
 - Define and understand tasks that require multimodal information to be solved
 - Understand how VLMs can be used to power multimodal agents
 - Overview of multimodal agentic benchmarks such as computer use agents
 - Open CUA: the first, fully open computer use agent pipeline to train a model for computer use

Moving beyond Text-only worlds

VISION-ENABLED AI AGENTS

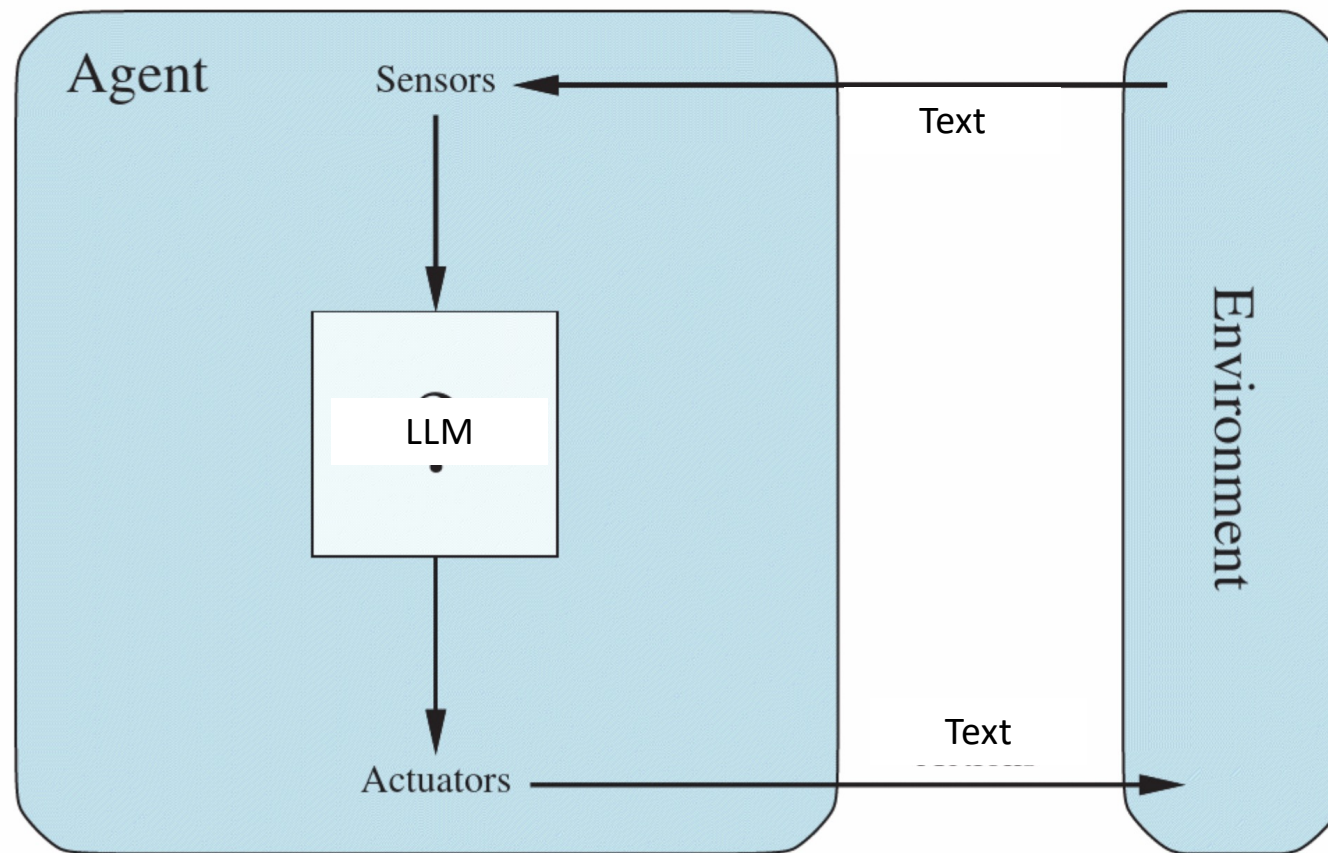
Recap: How do we define an agent?

“An agent is anything that can be viewed as perceiving its environment through sensors and acting upon that environment through actuators.”
-- Russell and Norvig



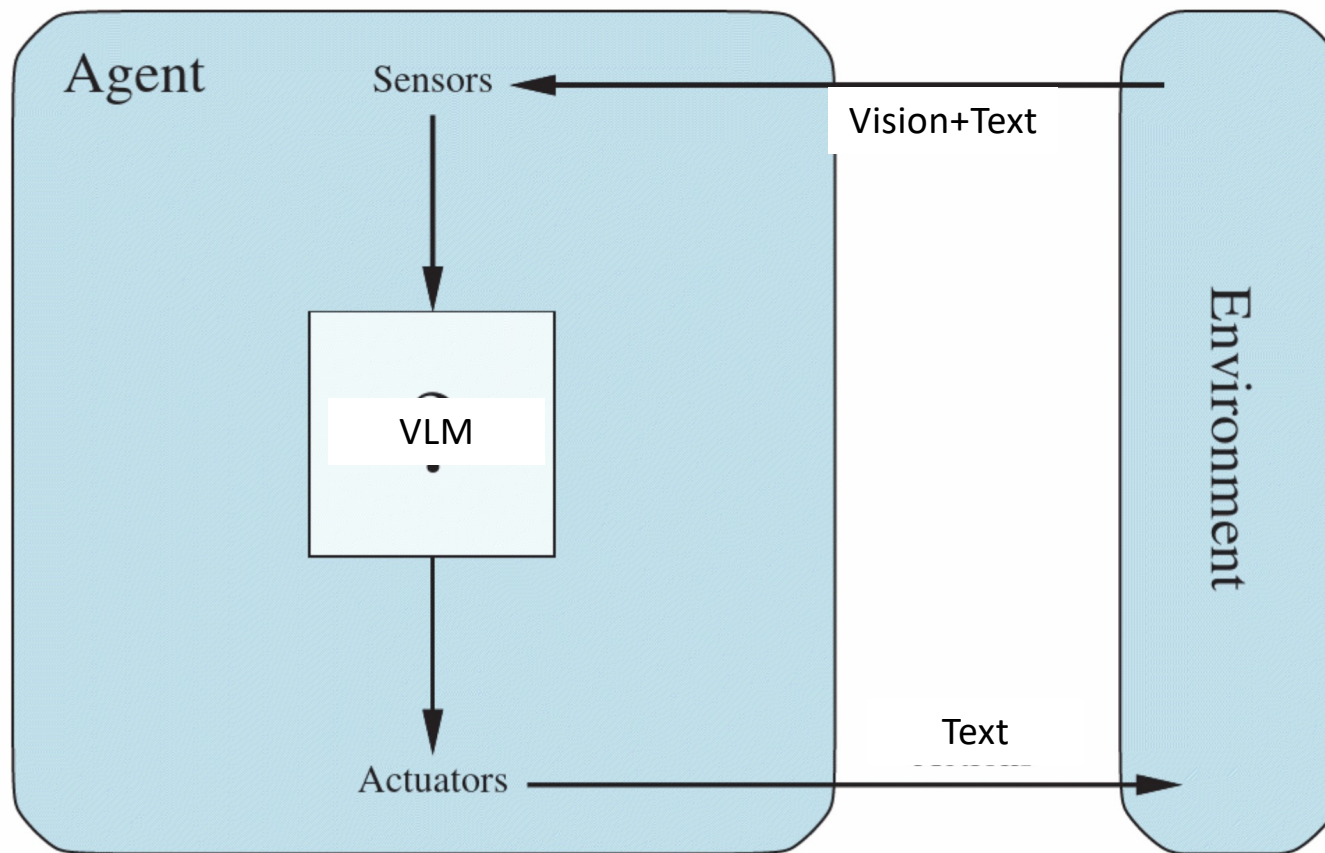
Stuart Russell and Peter Norvig. 2024. Artificial Intelligence: A modern Approach (4th. ed.). Prentice Hall Press, USA.

The Anatomy of a text-only LLM-based AI Agent



Main advantages: 1) leverage pretrained knowledge, 2) uses text to represent inputs/outputs

The Anatomy of a VLM-based AI Agent

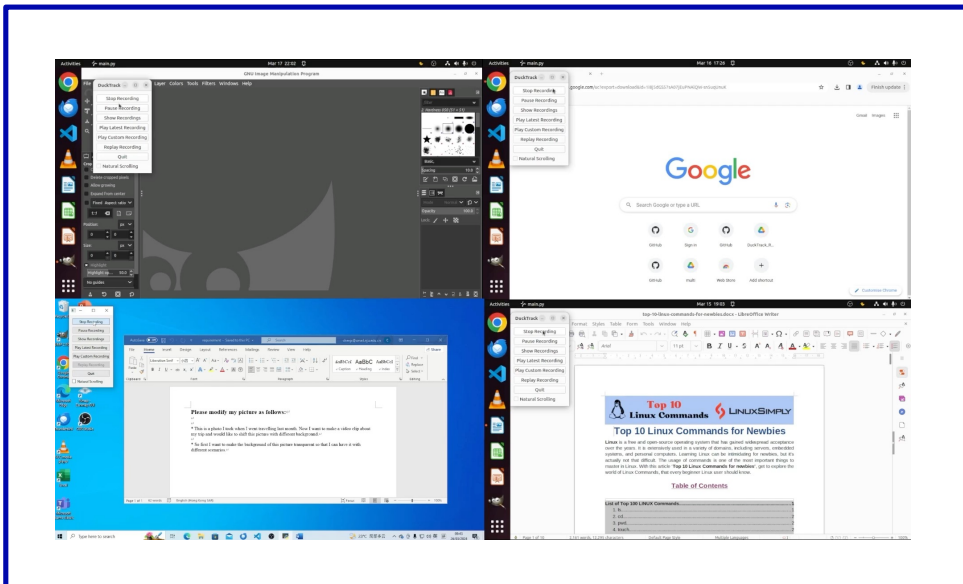


Benefits of a VLM-based AI Agent?

- In the previous lectures, we assumed that our agents would make decisions purely based on textual inputs
- Our world is inherently **multimodal**: our agents need to process much more than just textual inputs
 - Images
 - Videos
 - Graphs
 - Tables
- By leveraging a VLM, we can:
 1. Have access to pretrained knowledge available on the Web
 2. Use text/code to represent outputs
 3. Use vision directly avoiding context saturation; Why?

Embodied AI: AI Agents that interact with the world!

- With models that can perceive the world via the visual stream, we can enable a variety of different applications domains that go beyond simple text generation tasks
- Visually-enabled agents can now interact with both **simulated** and **real** worlds



In this lecture!

Visually-enabled AI agents that work in the operating system

COMPUTER USE AGENTS

Computer Use Agent (CUA)

- CUA are agentic solutions that **can control your computer** by interacting with all the applications available
- Given a language command, they **can derive a plan** to execute all the actions required to complete the task successfully
- This goes beyond searching for documents given a search query
 - By giving agents access to the OS applications, we can enable agents to perform complex workflows required for real world applications
 - It facilitates the execution on tasks on a variety of different platforms as well (e.g., different operating systems)
- In the limit, we would like to have fully autonomous agents that can **independently execute tasks on our behalf!**

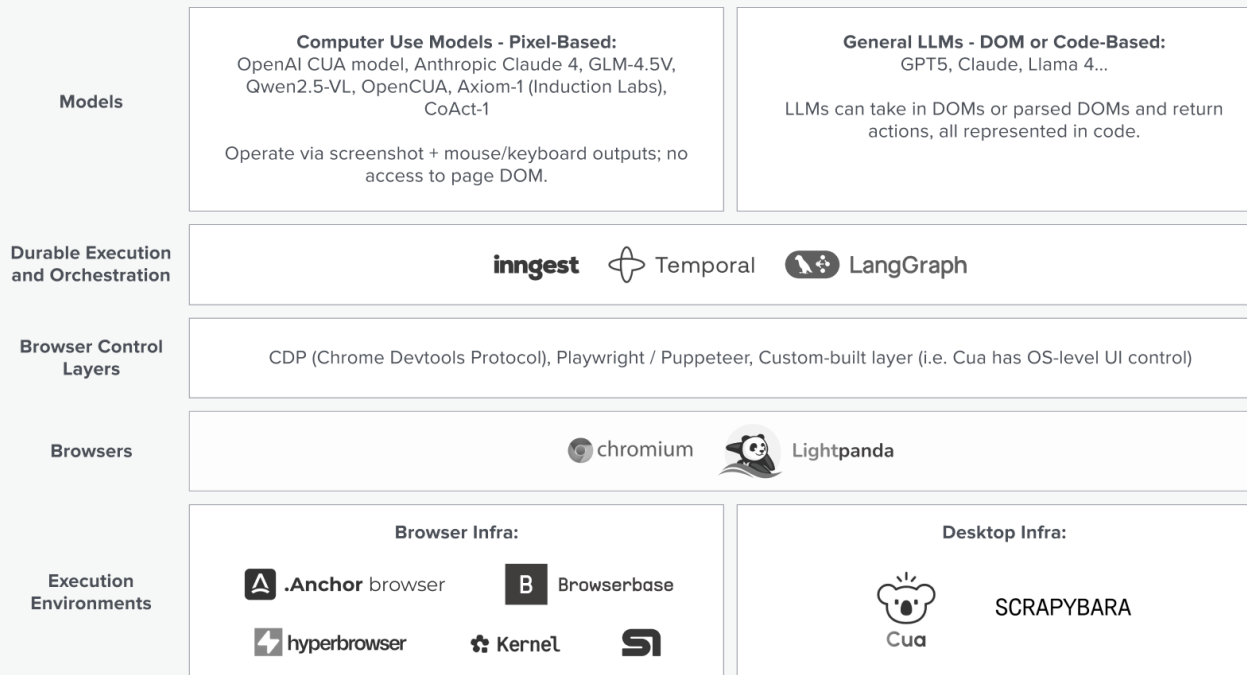
Constructing a Computer-Using Agent

Different ways to represent inputs!



Today we will focus on vision-based multimodal models!

Vision based ← → DOM based



Full stack applications:
 manus
Operator
SIMULAR

Some of the companies mentioned are portfolio companies of a16z. For a full list of portfolio companies, please visit a16z.com/investment-list.



<https://a16z.com/the-rise-of-computer-use-and-agentic-coworkers/>

Constructing a Computer-Using Agent

Different ways to represent inputs!

OmniParser
(model pipeline +
YOLO fine tune)

Browser Use (DOM +
optional vision with
Chrome Devtools
Protocol)

Cua
(DOM + vision)

Skyvern (vision
augmented,
LLM first)

Stagehand
(DOM, Playwright)

Today we will focus
on vision-based
multimodal models!

Vision based

DOM based

Very crowded opportunity space that
aims to revolutionise the way real-world
tasks are automated!

stack
cations:
nanus
erator
ULAR

Execution
Environments

 .Anchor browser

 Browserbase

 hyperbrowser

 Kernel

 SI

 Cua

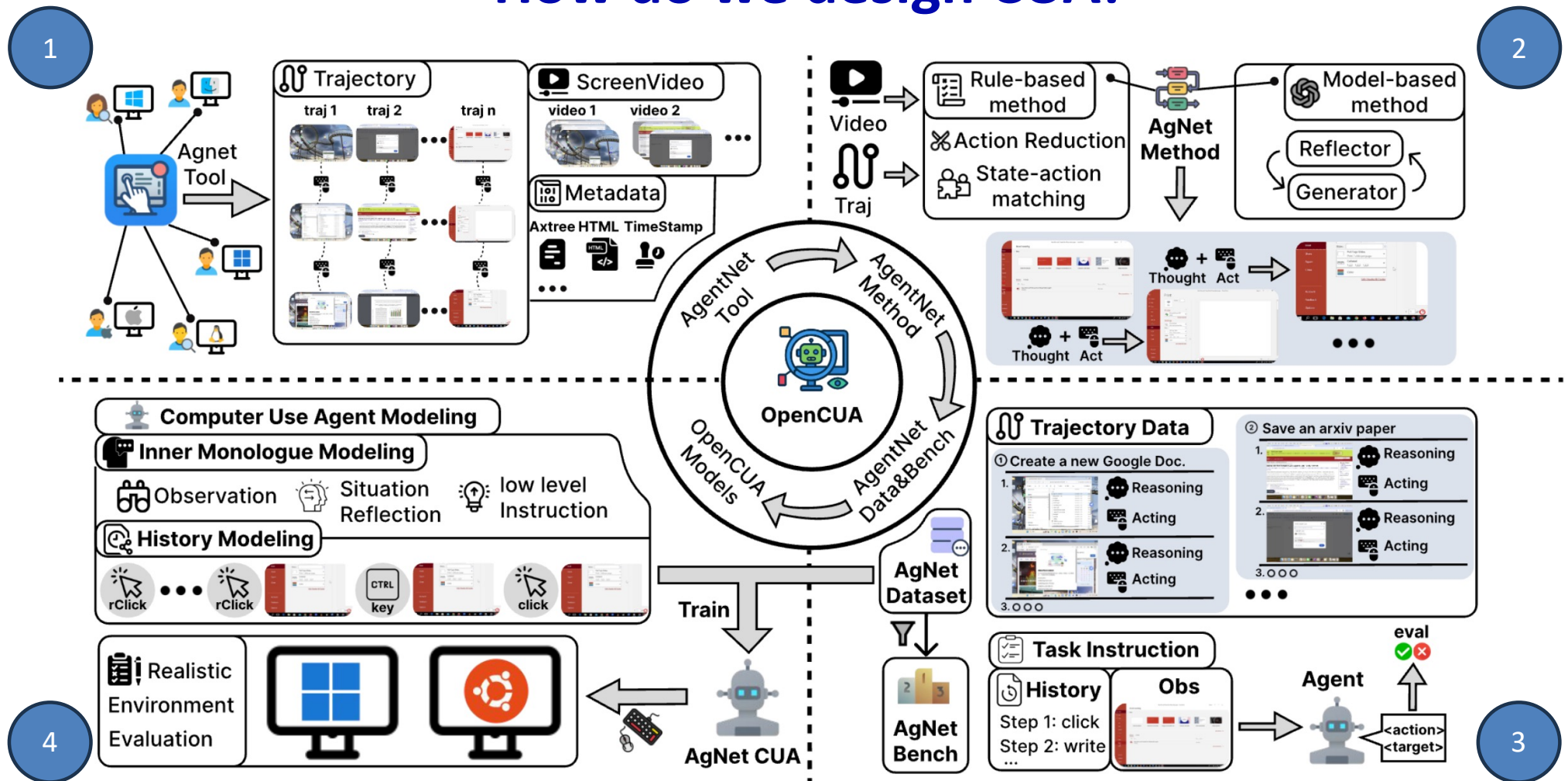
SCRAPYBARA

Some of the companies mentioned are portfolio companies of a16z. For a full list of portfolio companies, please visit a16z.com/investment-list.

 Enterprise

<https://a16z.com/the-rise-of-computer-use-and-agentic-coworkers/>

How do we design CUA?



Wang, Xinyuan, et al. "OpenCUA: Open Foundations for Computer-Use Agents." NeurIPS 2025.

The agent decision-making process

- We model the decision-making process of an agent as a **trajectory**:

$$(\mathcal{I}, \langle s_0, a_0 \rangle, \langle s_1, a_1 \rangle, \dots, \langle s_T, a_T \rangle)$$

- Where $\mathcal{I}$ is a language instruction or command for the agent (e.g., “find a pair of blue jeans on Amazon.co.uk”) and s_0 is the initial state of the agent
- The agent sequentially predicts an action a_i after *perceiving* the state of the environment s_i
- It does so until it reaches the goal state s_T and performs a termination action a_T
- **Vision-based agents**: We assume that the *state* is a screenshot of *the screen* that the agent can see.
 - ❑ No DOM or HTML of the page!

Wang, Xinyuan, et al. "OpenCUA: Open Foundations for Computer-Use Agents." NeurIPS 2025.

The action representation

- We model the decision-making process of an agent as a **trajectory**:

$$(\mathcal{I}, \langle s_0, a_0 \rangle, \langle s_1, a_1 \rangle, \dots, \langle s_T, a_T \rangle)$$

- Actions a_i are modelled as functions:

Human Action	Action Description	Agent Action
Click	Click at a specific position	click (x, y, button)
Middle Click	Middle click at a specific position	middleClick (x, y)
Double Click	Double click at a specific position	doubleClick (x, y, button)
Triple Click	Triple click at a specific position	tripleClick (x, y, button)
Mouse Move	Move mouse to a specific position	moveTo (x, y)
Drag	Drag mouse from one position to another	dragTo (x, y)
Scroll	Scroll vertically or horizontally	scroll (dx, dy) / hscroll (dx, dy)
Type	Type a string of text	write (text)
Press	Press a specific key	press (key)
Hotkey	Perform a combination of keys	hotkey (key1, key2)
Wait	Wait for a few seconds	wait ()
Terminate	End the task with success or failure	terminate ('success' or 'failure')

Wang, Xinyuan, et al. "OpenCUA: Open Foundations for Computer-Use Agents." NeurIPS 2025.

AgentNet Tool: Collecting Data for CUA

- AgentNet Tool provides a convenient way of automatically collecting interaction data of annotators performing tasks on a computer
- The tool automatically collects:
 - Screen recording using OBS Studio
 - Mouse and keyboard input tracking using DuckTrack and OpenAdapt
 - Accessibility trees
- Annotators were provided with a list of 200 applications and websites and had to complete a variety of different tasks with the aim to increase **diversity** and **complexity** of the tasks
- Data are collected across many operating systems including Ubuntu, Windows, and Mac
- All tasks were manually verified and labelled as **rejected**, **ok**, **good**, or **excellent** based on goal clarity, diversity, and complexity

Wang, Xinyuan, et al. "OpenCUA: Open Foundations for Computer-Use Agents." NeurIPS 2025.

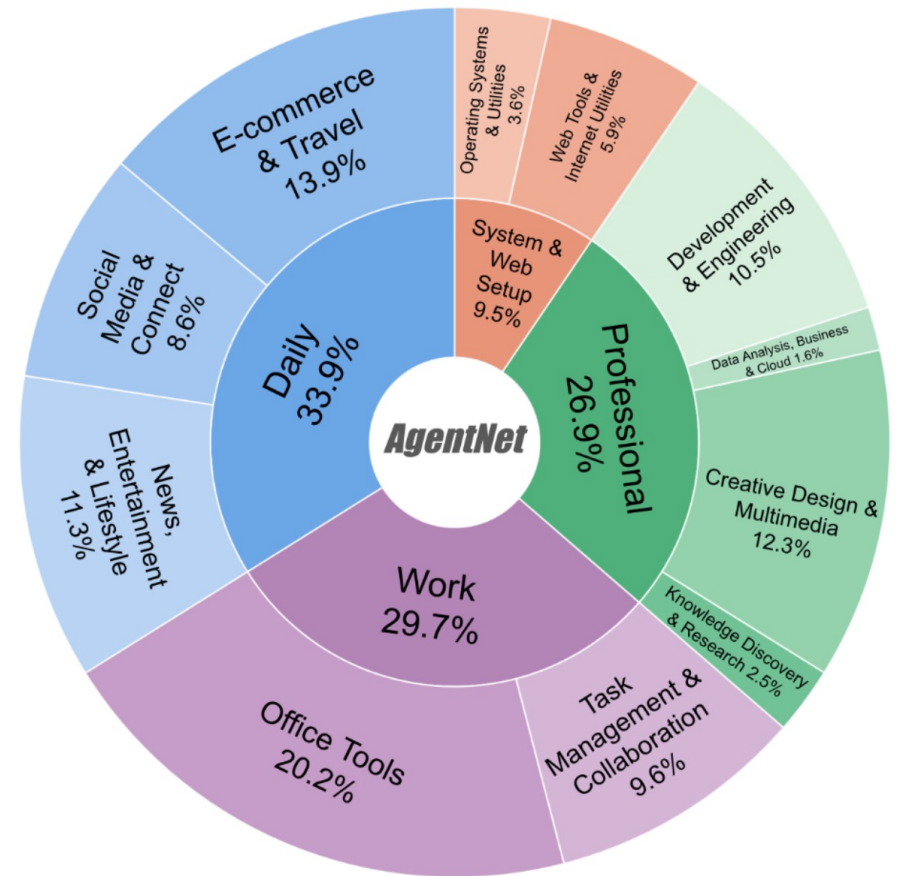
How do we curate the data?

- Remember that AgentNet Tool collects directly screen recordings which have high-frequency frame rates associated with real humans completing tasks
 - ❑ Can our agents learn from this video data? No!
- Therefore, the authors created a careful data processing pipeline that ensures
 1. Action reduction: reduce the density of actions and focus on high-level signals only (e.g., for “scroll” actions we record only the start and end position)
 2. State-action matching: extract key video frames that are useful to predict the action at the current timestep
- After the final action, they append a final termination frame that determines the last state of the agent

Wang, Xinyuan, et al. "OpenCUA: Open Foundations for Computer-Use Agents." NeurIPS 2025.

AgentNet dataset

- The dataset contains 22,625 human-annotated computer use trajectories including:
 - 12K from Windows
 - 5K from macOS
 - 5K from Ubuntu
- Different screen resolutions supported ranging from 720p to 4K
- Each trajectory averages 18.6 steps!
- Models have to be able to encode the history of the trajectory to make sense of potential mistakes or state transitions



Wang, Xinyuan, et al. "OpenCUA: Open Foundations for Computer-Use Agents." NeurIPS 2025.

How do we train the agent?

- The AgentNet dataset provides us with a good number of agent trajectories that we can use to train our CUA
 - Can we finetune a VLM on this data? Yes!
- For instance, let's assume we will be using [SmolVLM](#) (ATNLP Lecture 26!). How do we do this?
- We have trajectories composed of a language instruction and state-action pairs: $(\mathcal{I}, \langle s_0, a_0 \rangle, \langle s_1, a_1 \rangle, \dots, \langle s_T, a_T \rangle)$
- We can use $(\mathcal{I}, \langle s_i, a_i \rangle)$ as training data for our VLM where we assume that the action is represented as text and the state is an image
- The authors of OpenCUA tried this approach and it didn't work well!
 - Why? Inability to ground and reason about the task and the trajectory!

Wang, Xinyuan, et al. "OpenCUA: Open Foundations for Computer-Use Agents." NeurIPS 2025.

Long-form Chain-of-Thought for CUA

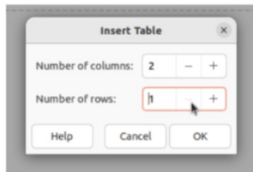
- To incentivise the model to reason about the task and what it sees on the screen, they synthesise a dataset that including long-form Chain-of-Thought data
- They structure their approach in three different levels:
 - Level 3: focus on salient textual and visual elements
 - Level 2: reflective reasoning focusing on planning next actions, reflecting on previous steps
 - Level 1: model predicts the action which is now grounded in the reasoning trace as well
- They design a synthetic generation pipeline by using Claude-3.7 Sonnet

Wang, Xinyuan, et al. "OpenCUA: Open Foundations for Computer-Use Agents." NeurIPS 2025.

Long-form Chain-of-Thought for CUA

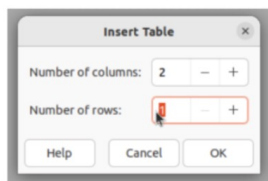
Task: In the "Features" slide, insert a table with 5 rows and 2 columns.

Step 8: Click on the "Number of rows:" input field to ensure it's selected for editing.

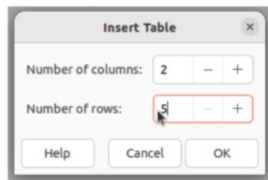


✗ Error:
Model mistakenly click the wrong place changing the rows from 2 to 1, instead of 5

Step 9: Click and drag across the "1" in the Number of rows field to select it for replacement.



Step 10: Type "5" to replace the selected "1" in the Number of rows field.



✓ Model's mistake is correctly resolved

Step 9 Long CoT

Reflection

I can see that we've made progress in setting up the table for the Features slide ... which aligns with our goal. **However, the number of rows is currently set to 1, but we need 5 rows according to our goal.**

Memory

Previously, we've already:

1. Navigated to slide 3 (the Features slide)

...

5. Set the number of rows to 1 (though we need to change this to 5)

Plan & Reasoning

The current state shows that we're in ... showing "1". Since we need 5 rows instead of 1, we need to change this value. **We could select the current value and replace it with "5"** After selecting the "1", the next logical step would be to type "5" to replace it, and then click the OK button to create the table with our desired dimensions (5 rows and 2 columns).

Prediction

Drag to select the number of rows "1".

Code

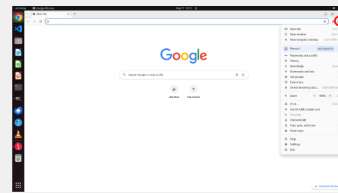
```
pyautogui.moveTo(1362, 578); pyautogui.dragTo(1320, 578);
```

The OpenCUA Training Recipe

- To support their long-form CoT data, the authors designed a careful input format as well as data mixture to support different basic capabilities
- This includes different reasoning traces formats (L1, L2, L3) as well as different tasks.
 - L1 = action only
 - L2 = thought + action
 - L3 = observation + thought + action
- They also include additional inputs such as:
 - Action history: Previous actions formatted using L1
 - Visual history: Previous three screenshots
- Always consider the trade-off between **accuracy/inference speed!**

Wang, Xinyuan, et al. "OpenCUA: Open Foundations for Computer-Use Agents." NeurIPS 2025.

- To support the different input format as well as the different capabilities
- This includes different tasks
 - L1 = action
 - L2 = thought
 - L3 = observation
- They also include
 - Action hierarchy
 - Visual hierarchy
- Always consider



Step 1

Thought: I need to install a Chrome extension that's located in the Desktop directory. Looking at the current state, I'm on the Google homepage in Chrome. To install an extension, I need to access the Chrome Web Store or use the extensions menu.

I can see that there's a "Web Store" shortcut on the Google homepage which could be useful. However, I also notice the Chrome menu button (three vertical dots) in the top-right corner of the browser. This menu would give me access to more Chrome settings and features, including extensions.

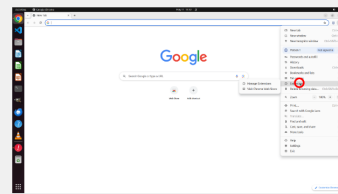
Since I need to install a local extension file, the most direct approach would be to: 1. Click on the Chrome menu (three dots) 2. Navigate to "More tools" > "Extensions" or directly to "Chrome Web Store" 3. Upload the extension file from the Desktop.

The Chrome menu button in the top-right corner is the logical next action because it provides access to all Chrome settings and features, including extensions management. Clicking this button will open a dropdown menu with various options, including those related to extensions.

I anticipate that after clicking the menu button, I'll see options that will allow me to navigate to the extensions page or Chrome Web Store, where I can then upload the extension file from the Desktop.

Action: Click on the three-dot menu button in the top-right corner of the Chrome browser window to open the Chrome menu.

Code: `pyautogui.click(x=0.988, y=0.081)`



Step 2

Thought: After clicking on the three-dot menu in the previous step, the Chrome menu has successfully opened, displaying various options. This is exactly what I expected to happen, so we're making good progress toward our goal of installing a Chrome extension from the Desktop. Looking at the current state, I can see that the Chrome menu is now open with all its options. To install a Chrome extension, I need to access the Extensions page or use the Web Store. Based on the goal of installing an extension from the Desktop directory, I should look for the Extensions option in this menu.

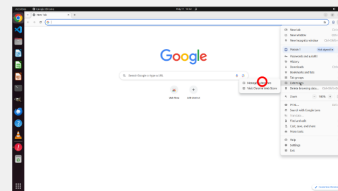
I can see that there is an "Extensions" option in the menu, which is exactly what I need. Clicking on this option will take me to Chrome's Extensions page, where I can then load an unpacked extension from the Desktop directory.

The logical next action would be to click on the "Extensions" option in the Chrome menu. This will navigate to the Extensions management page where I can then find an option to load an unpacked extension. From there, I'll be able to browse to the Desktop directory and select the extension file for installation.

By clicking on the Extensions option, I expect to be taken to the Extensions management page where I'll see a list of currently installed extensions and options to manage them, including an option to "Load unpacked" which I'll need to use to install the extension from the Desktop.

Action: Click on the "Extensions" option in the Chrome menu to access the extensions management page.

Code: `pyautogui.click(x=0.867, y=0.406)`



Step 3

Thought: After clicking on the "Extensions" option in the Chrome menu, I can see that the menu has expanded to show more options related to extensions. This is the expected result of the previous action.

Looking at the current state, I can see that there are two main options related to extensions: 1. "Manage Extensions" - This would take me to the page where I can manage currently installed extensions 2. "Visit Chrome Web Store" - This would take me to the Chrome Web Store where I can download extensions from the official store

Since our goal is to install a Chrome extension that is located on the Desktop directory, I need to access the extensions management page first. From there, I would be able to access the "Load unpacked" button which allows installing extensions from local files rather than from the Chrome Web Store.

The logical next action would be to click on "Manage Extensions" since this is the option that will take me to the extensions management page where I can install the extension from the Desktop. This is the most direct path to achieving our goal of installing a locally developed extension.

By clicking on "Manage Extensions," I expect to be taken to a page that shows all currently installed extensions and provides options to add new extensions, including the "Load unpacked" option that I'll need to use to install the extension from the Desktop directory.

Action: Click on the "Manage Extensions" option in the Chrome menu to access the extensions management page.

Code: `pyautogui.click(x=0.755, y=0.403)`

a careful input
capabilities
as well as

speed!

The OpenCUA Training Recipe

- They use a QwenVL-2.5 model as starting point for their experiments; They finetune three different model sizes:
 - 7B
 - 32B
 - 72B
- They explored different training regimes that use data during different phases
 - Stage 1 = visual grounding on GUI screens
 - Stage 2 = planning + reasoning in OS environment
- The authors report additional results for other ways of combining these two stages
 - It turns out that **joint training** is the best approach but more computationally intensive!

Wang, Xinyuan, et al. "OpenCUA: Open Foundations for Computer-Use Agents." NeurIPS 2025.

The OpenCUA Training Recipe

- **Stage 2 only:** the most interesting setting for a low-compute regime where you finetune a pretrained VLM using trajectory data directly
- They trained on 18k Win&macOS + 10k Ubuntu trajectories.
 - OPENCUA-QWEN2-7B runs for 3,400 steps (about 45 h) after a 400-step grounding warm-up
 - OPENCUA-A3B runs for 2,000 steps (about 10 h)
- More modern VLMs are now **already trained** using grounding data collected on GUI screens and trajectories so this adaptation pipeline would be even more effective for specific downstream tasks

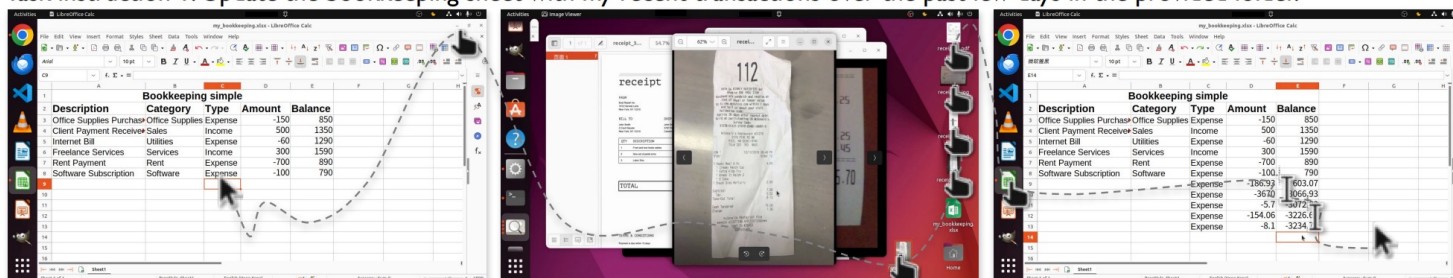
Wang, Xinyuan, et al. "OpenCUA: Open Foundations for Computer-Use Agents." NeurIPS 2025.

An overview of benchmarks and evaluation metrics

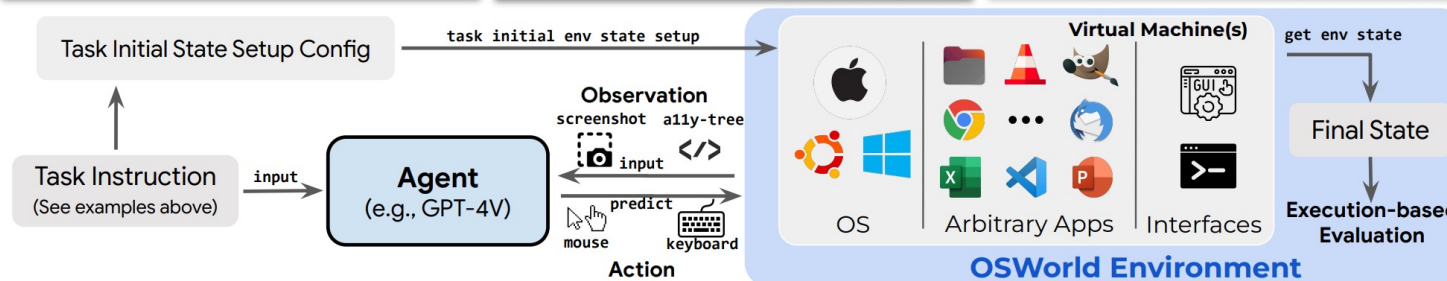
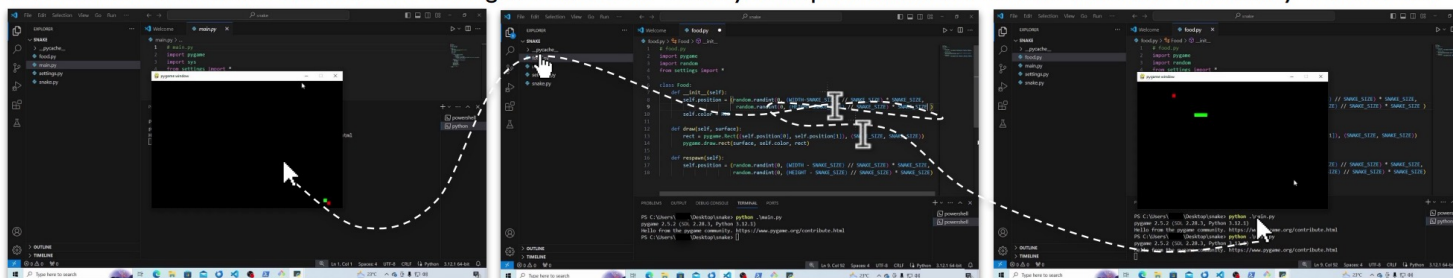
EVALUATING COMPUTER USE AGENTS

OSWorld

Task instruction 1: Update the bookkeeping sheet with my recent transactions over the past few days in the provided folder.



Task instruction 2: ...some details about snake game omitted... Could you help me tweak the code so the snake can actually eat the food?

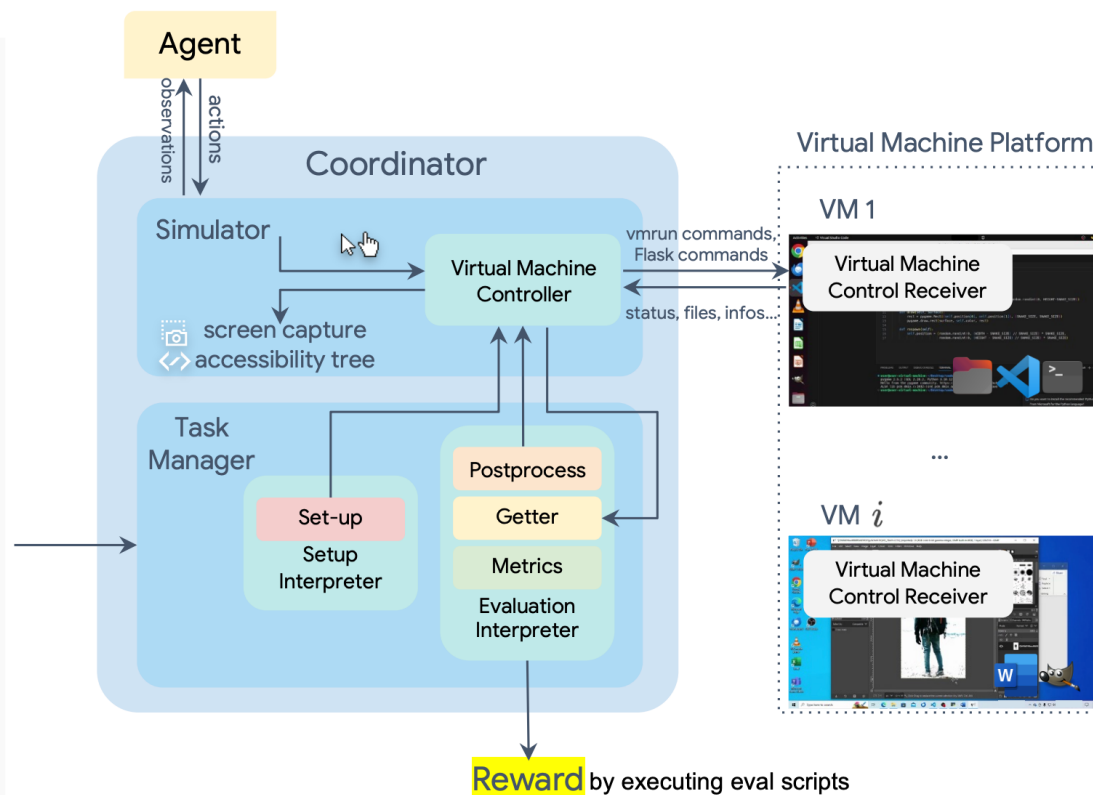


Xie, Tianbao, et al. "Osworld: Benchmarking multimodal agents for open-ended tasks in real computer environments." NeurIPS 2024.

OSWorld

Config

```
{ "instruction": "Please update my bookkeeping sheet with
the recent transactions from the provided folder, detailing
my expenses over the past few days.",
  "config": [{ "type": "download",
    "parameters": { "files": [
      { "path": "/home/user/Desktop/my_bookkeeping.xlsx",
        "url": "https://drive.google.com/uc?id=xxxx" },
      { "path": "/home/user/Desktop/receipt_0.jpeg",
        "url": "https://drive.google.com/uc?id=xxxx" }, ... ] } },
    { "type": "open",
      "parameters": { "path":
"/home/user/Desktop/my_bookkeeping.xlsx" } } ],
  "evaluator": { "postconfig": [{ "type": "activate_window",
    "parameters": { "window_name": "my_bookkeeping.xlsx -
LibreOffice Calc", ... } },
    { "result": { "type": "vm_file",
      "path": "/home/user/Desktop/my_bookkeeping.xlsx",
      "dest": "my_bookkeeping.xlsx" },
      "expected": { "type": "cloud_file",
        "path": "https://drive.google.com/uc?id=xxx",
        "dest": "my_bookkeeping_gold.xlsx" },
      "func": "compare_table",
      "options": {
        "rules": [{
          "type": "sheet_fuzzy",
          "sheet_idx0": "RNSheet1",
          "sheet_idx1": "ENSheet1",
          "rules": [ { "range": [ "A1:A8", ... ] } ] } ] } } ] } }
```



Xie, Tianbao, et al. "Osworld: Benchmarking multimodal agents for open-ended tasks in real computer environments." NeurIPS 2024.

OSWorld-Verified

- Since the initial release in 2024, OSWorld has become an industry standard for CUA evaluation with many giants including OpenAI and Anthropic using it as a benchmark environment for their frontier models
- However, people have identified important weaknesses of the original data collection that have had an effect on model performance
- **OSWorld-Verified** is the current instantiation of the benchmark which includes:
 1. Infrastructure: Migrated to AWS cloud for massive parallelization (10+ hours → minutes)
 2. Task Quality: Fixed 300+ issues including web structure changes, instruction ambiguity, and evaluation robustness
 3. Evaluation: Established public evaluation platform for verified apple-to-apple comparisons
 4. Performance: Current leading models show success primarily stems from extensive human trajectory data

Xie, Tianbao, et al. "Osworld: Benchmarking multimodal agents for open-ended tasks in real computer environments." NeurIPS 2024.

Important changes in OSWorld-Verified

It is interesting to verify the exact issues that the authors have identified in OSWorld. Some of them are reported below:

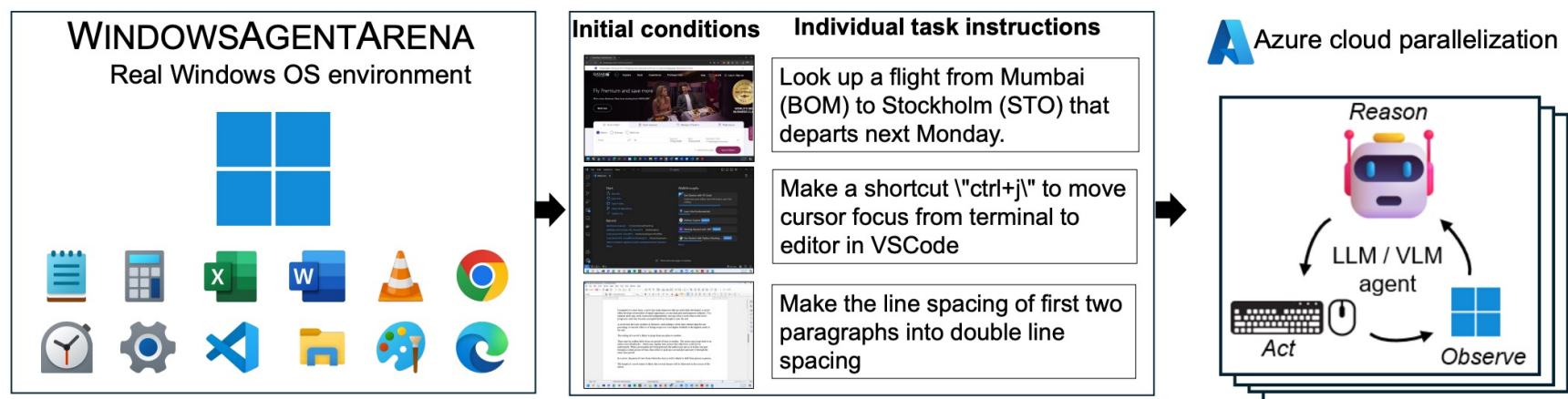
- Web Structure and Content Changes: Websites frequently change their HTML structure, CSS classes, and content, causing evaluation functions to fail or become unreliable.
- Time-Sensitive and Booking Tasks: Tasks involving future dates, reservations, or time-sensitive content become invalid over time.
- Instruction Ambiguity and Clarity: Vague or ambiguous instructions leading to multiple valid interpretations and evaluation inconsistencies.
- Evaluation Function Robustness: Overly strict or inadequate evaluation functions causing incorrect scoring. Very hard to consider different file formats at scale!

More information on this update can be found on their [changelog](#)

OSWorld-Verified: <https://xlang.ai/blog/osworld-verified>

WindowsAgentArena

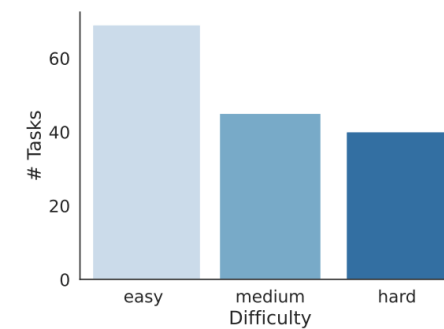
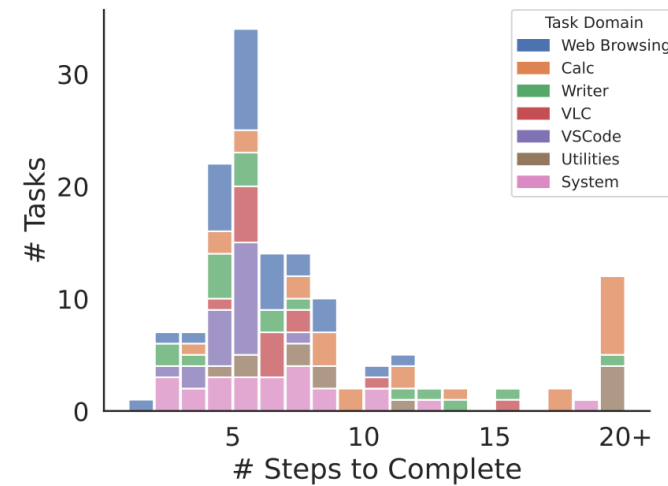
- Designed with the same setup of OSWorld but with some extra optimisations that natively leverage the ability of Azure Cloud to parallelise virtual environments in a very fast way
- This makes it possible to train RL agents by spawning multiple instances with different tasks and environments specifications!



Bonatti, Rogerio, et al. "Windows agent arena: Evaluating multi-modal os agents at scale." arXiv preprint arXiv:2409.08264 (2024).

WindowsAgentArena

Domain	Activity Examples	# Tasks	%
Office	Libreoffice Writer (Editing, writing) and LibreOffice Calc (spreadsheets, plotting, data manipulation)	43	28%
Web Browsing	Online shopping & search, customizing app settings with Edge and Chrome	30	19%
Windows System	File Explorer (navigate, customize files), Windows Settings, customization	24	16%
Coding	Editing code, customizing IDE, installing extensions with VSCode	24	16%
Media & Video	Watching videos in VLC, listening to music, settings	21	14%
Windows Utilities	Miscellaneous tasks with Notepad, Clock, and Paint	12	8%
Total		154	100%



Bonatti, Rogerio, et al. "Windows agent arena: Evaluating multi-modal os agents at scale." arXiv preprint arXiv:2409.08264 (2024).

ScienceBoard

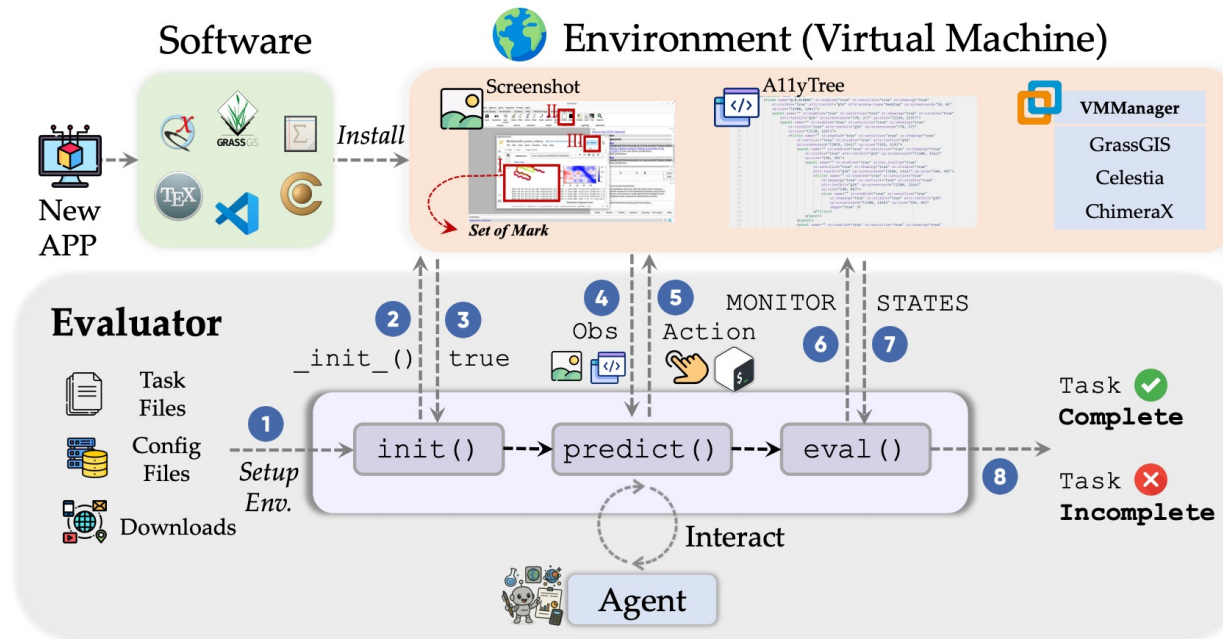


Figure 2: Overview of the SCIENCEBOARD infrastructure. The scalable environment is built upon a VM pre-installed with scientific discovery software. It supports both CLI and GUI interfaces to enable autonomous agent interaction. For each task designed to evaluate the agent's capability as a research assistant, an initialization script, configs, and related files are provided. Agents perceive the environment through visual or textual modalities, and are expected to plan and act accordingly. After the interaction, an evaluation function determines completion based on the VM internal states.

Sun, Qiushi, et al. "Scienceboard: Evaluating multimodal autonomous agents in realistic scientific workflows." arXiv preprint arXiv:2505.19897 (2025).

Online-Mind2Web

- Benchmark created to assess the ability of agents to perform tasks on **real-world websites** under a setting that approximate how real users would use them
- The benchmark was created from Mind2Web by:
 1. Considering only live websites rather than cached versions
 2. Filtering out tasks with unclear or vague instructions
 3. Removing invalid tasks that are not executables due to changes to the websites
 4. Removing CAPTCHA-protected websites that prevent access to agents



Sun, Qiushi, et al. "Scienceboard: Evaluating multimodal autonomous agents in realistic scientific workflows." arXiv preprint arXiv:2505.19897 (2025).

How do we evaluate CUA?

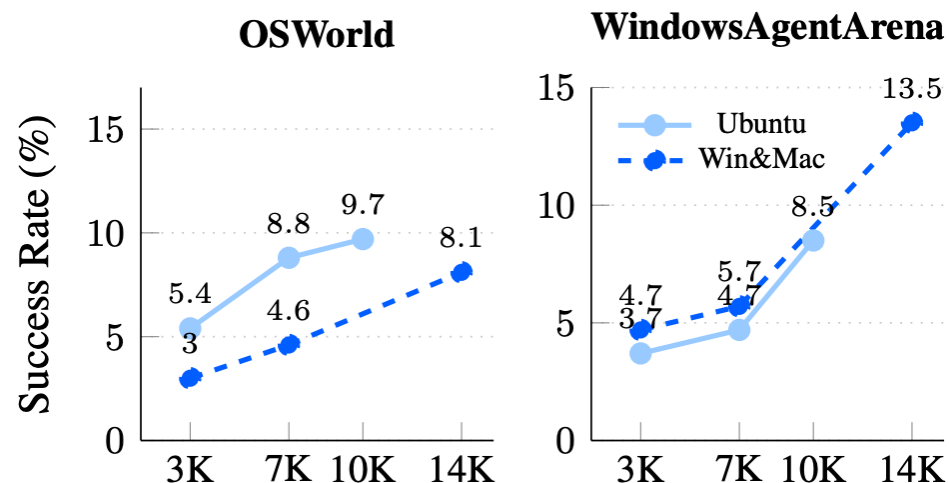
- Evaluation of CUA agents requires assessing whether the agent can reach the final goal given a language e instruction
- Success rate metrics are useful to determine how often agents can successfully complete the tasks
- It is also important to use specialised datasets to assess the ability to perform **visual grounding** (e.g., select a given button!)
- However, CUA tasks typically involve a very high number of steps which complicates the evaluation setup. Many important aspects to think about:
 - How do we reward intermediate goals?
 - How do we recognise when the agent is stuck in a loop?
 - How do we incentivise the agent's ability to backtrack?
 - If the agent fails, is it because of its inability to execute actions or due to its inability to locate specific elements of the desktop?

Wang, Xinyuan, et al. "OpenCUA: Open Foundations for Computer-Use Agents." NeurIPS 2025.

OpenCUA evaluation

Model	15 Steps	50 Steps	100 Steps
Proprietary			
OpenAI CUA [31]	26.0	31.3	31.4
Seed1.5-VL [18]	27.9	-	34.1
Claude 4 Sonnet [4]	<u>31.2</u>	<u>43.9</u>	41.5
Claude Sonnet 4.5 [5]	-	-	<u>61.4</u>
Open-Source			
Qwen2.5-VL-32B-Instruct [7]	3.0	-	3.9
Qwen2.5-VL-72B-Instruct [7]	4.4	-	5.0
Kimi-VL-A3B [37]	9.7	-	10.3
UI-TARS-72B-DPO [32]	24.0	25.8	27.1
Qwen3-VL [7]	-	-	38.1
OpenCUA-7B (Ours)	24.3 ^{+1.9} _{-1.3}	28.1 ^{+0.7} _{-0.4}	26.6 ^{+0.6} _{-0.5}
OpenCUA-32B (Ours)	29.7 ^{+0.8} _{-1.5}	34.1 ^{+1.0} _{-0.6}	34.8 ^{+0.9} _{-1.0}
OpenCUA-72B (Ours)	39.0	44.9	45.0 ^{+1.1} _{-1.2}

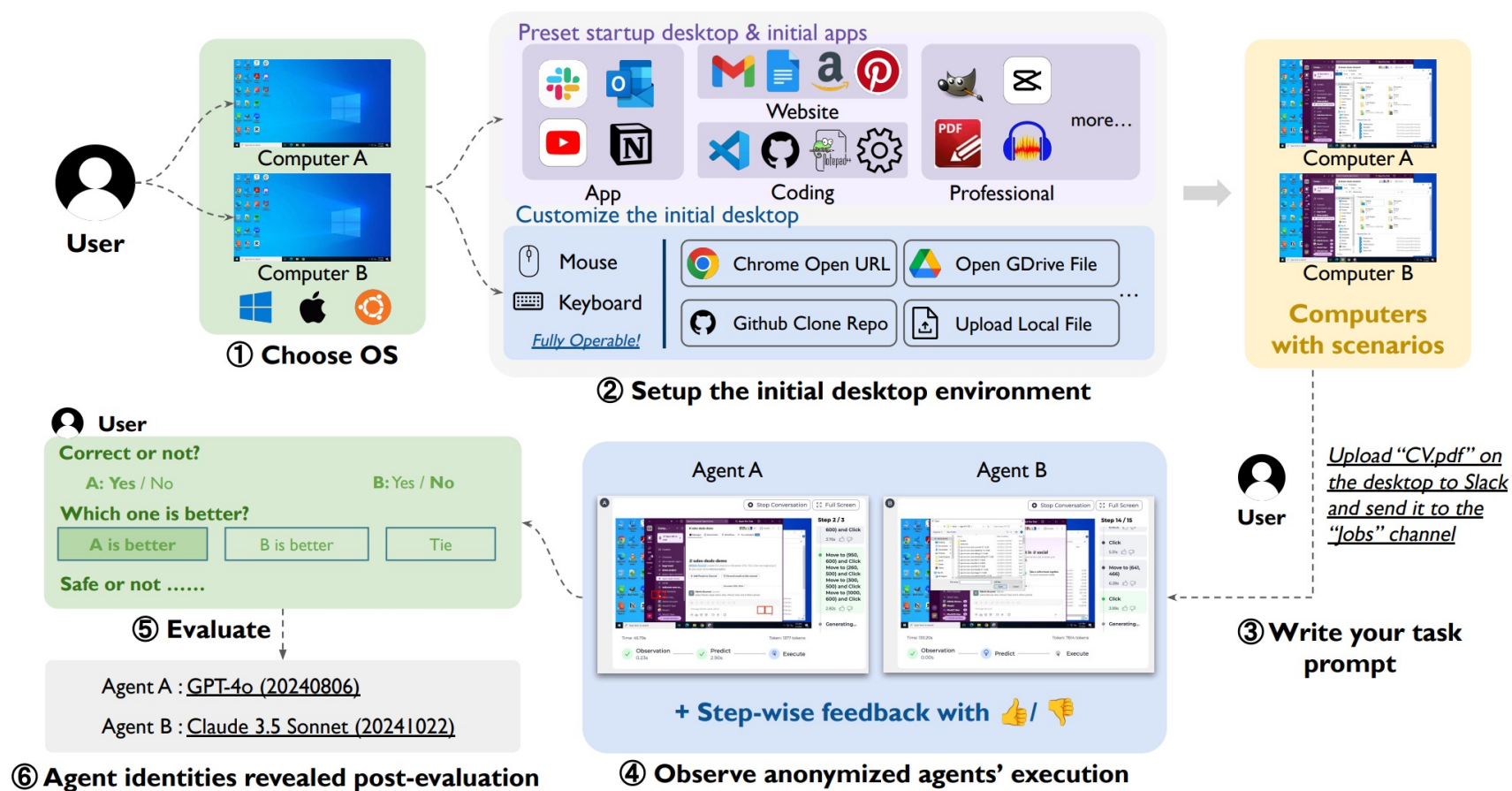
Fully open-source pipeline to build a state-of-the-art CUA from scratch. However, still lags behind proprietary models!



Performance increases by scaling the number of trajectories with behaviour that depends on OS.

Wang, Xinyuan, et al. "OpenCUA: Open Foundations for Computer-Use Agents." NeurIPS 2025.

ComputerUseArena: Evaluating CUA in the wild



Wang, Bowen, et al. "Computer Agent Arena: Toward Human-Centric Evaluation and Analysis of Computer-Use Agents." ICLR 2026

Why open-ended evaluation is important for CUA?

- [ComputerUseArena](#) offers a playground to compare models on a variety of different tasks that are not predefined by the evaluators
- Users can refer to any task and use any language instruction when formulating their request. This allows to test [the robustness of the models](#)
- Authors find an interesting trend:
 - The resulting leaderboard diverges sharply from static benchmarks—most notably, several top performers on OSWORLD are inverted in their setting. This highlights an **important lack of robustness!**

Rank	Model	Elo	Votes	Correct Rate
1	Claude Sonnet 4	1167	416	52.0%
2	Claude 3.7 Sonnet	1140	507	52.3%
3	UI-TARS-1.5	1092	533	49.9%
4	Operator	1064	511	37.4%
5	CoAct-1*	1043	110	41.8%
6	OpenCUA*	1023	109	38.5%
7	Claude 3.5 Sonnet	1023	425	35.8%
8	GPT-5*	1002	108	34.3%
9	o4-mini	895	266	15.4%
10	Qwen 2.5 VL 72B Instruct	895	504	15.9%
11	GPT-4.1	837	432	8.6%
12	Gemini 2.5 Pro	829	377	11.8%

(a) COMPUTER AGENT ARENA leaderboard with Elo scores, vote counts, and correctness rates.

Future work on CUA

- An incredible opportunity space for researchers and practitioners to redefine the way we interact with computers and define computer-based workflows
- However, for production-ready systems we need to ensure reliability across a variety of axes:
 1. The visual grounding gap
 2. Long-horizon modelling
 3. Credit-assignment problem
 4. Security & Reliability

Wang, Xinyuan, et al. "OpenCUA: Open Foundations for Computer-Use Agents." NeurIPS 2025.

Future work on CUA

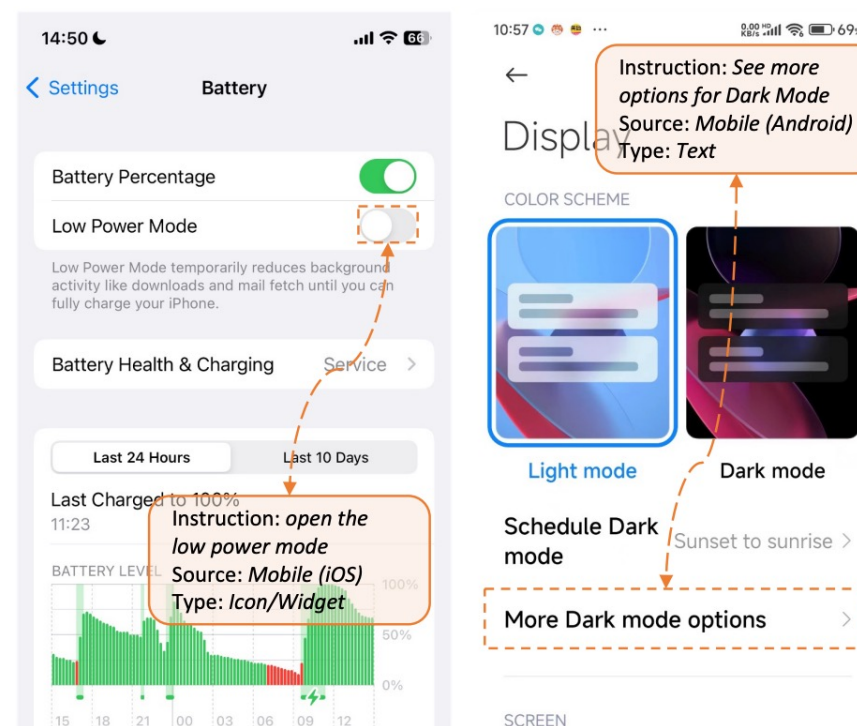
- An incredible opportunity space for researchers and practitioners to redefine the way we interact with computers and define computer-based workflows
- However, for production-ready systems we need to ensure reliability across a variety of axes:
 1. The visual grounding gap
 2. Long-horizon modelling
 3. Credit-assignment problem
 4. Security & Reliability

Wang, Xinyuan, et al. "OpenCUA: Open Foundations for Computer-Use Agents." NeurIPS 2025.

Future work on CUA

- An incredible opportunity space for researchers and practitioners to redefine the way we interact with computers and define computer-based workflows
- However, for production-ready systems we need to ensure reliability across a variety of axes:
 1. The visual grounding gap
 2. Long-horizon modelling
 3. Credit-assignment problem
 4. Security & Reliability

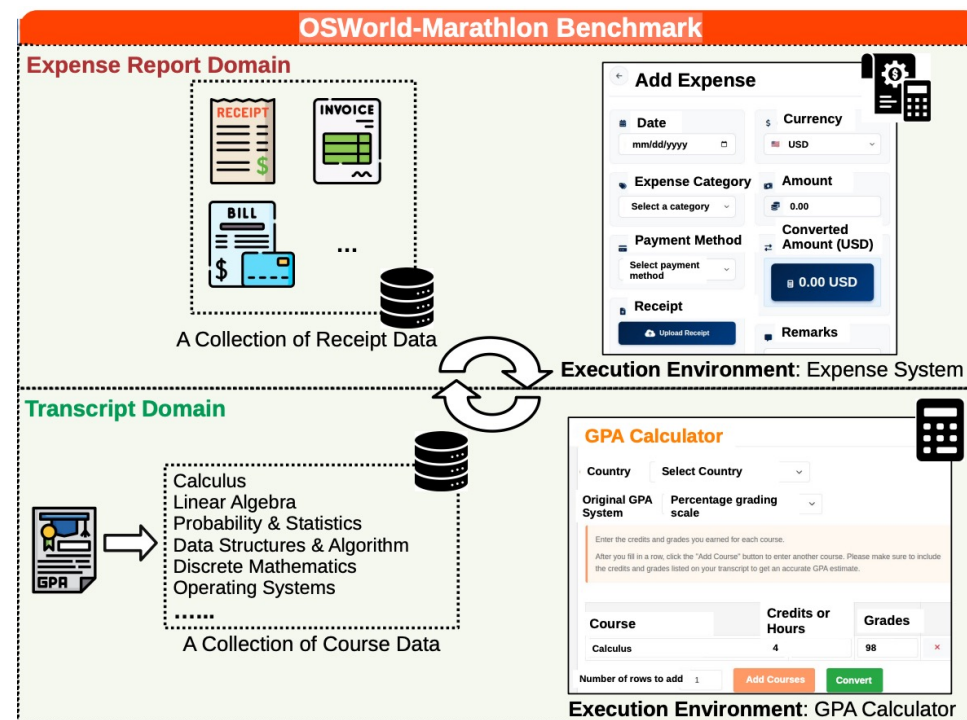
GUI Grounding Benchmark: *ScreenSpot*



Cheng, Kanzhi, et al. "Seedclick: Harnessing gui grounding for advanced visual gui agents." ACL 2024.

Future work on CUA

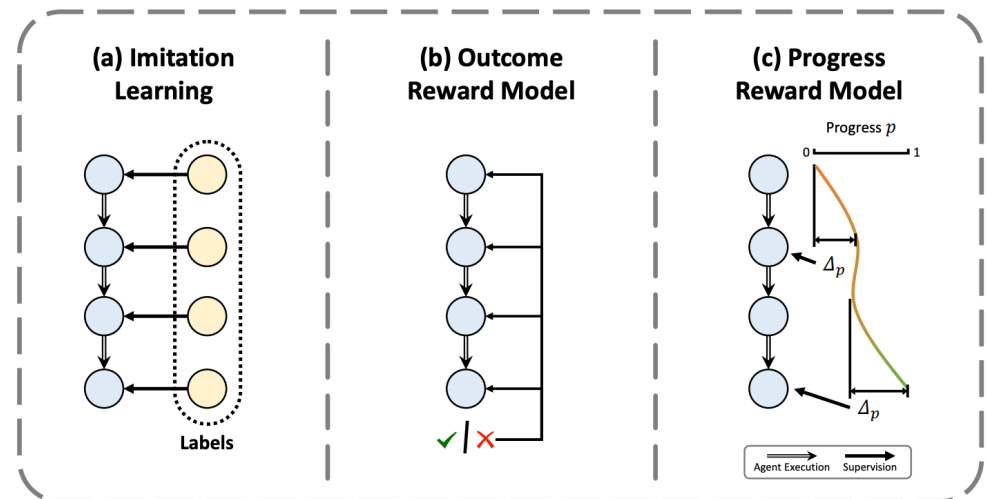
- An incredible opportunity space for researchers and practitioners to redefine the way we interact with computers and define computer-based workflows
- However, for production-ready systems we need to ensure reliability across a variety of axes:
 1. The visual grounding gap
 2. Long-horizon modelling
 3. Credit-assignment problem
 4. Security & Reliability



Wu, Jing, et al. "OS-Marathon: Benchmarking Computer-Use Agents on Long-Horizon Repetitive Tasks." arXiv preprint arXiv:2601.20650 (2026).

Future work on CUA

- An incredible opportunity space for researchers and practitioners to redefine the way we interact with computers and define computer-based workflows
- However, for production-ready systems we need to ensure reliability across a variety of axes:
 1. The visual grounding gap
 2. Long-horizon modelling
 3. Credit-assignment problem
 4. Security & Reliability



Zhang, Danyang, et al. "Progrm: Build better gui agents with progress rewards." arXiv preprint arXiv:2505.18121 (2025).

Future work on CUA

- An incredible opportunity space for researchers and practitioners to redefine the way we interact with computers and define computer-based workflows
- However, for production-ready systems we need to ensure reliability across a variety of axes:
 1. The visual grounding gap
 2. Long-horizon modelling
 3. Credit-assignment problem
 4. Security & Reliability

(c) The agent is distracted by the environment.



Ma, Xinbei, et al. "Caution for the Environment: Multimodal LLM Agents are Susceptible to Environmental Distractions." arXiv preprint arXiv:2408.02544 (2024).

Summary

- We investigated how to go beyond agents that are purely observing textual inputs.
- Enabling agents to perceive the world via visual stimuli enables a variety of different application domains ranging from computer use agents to robotics
 - Language becomes the bridge between high-level planning and low-level action execution!
- We focused on Computer Use Agents (CUA) as an instance of visually-enabled agents that can operate within an operating system by clicking buttons and using applications
- We studied OpenCUA as an open implementation of a state-of-the-art foundation model for computer use for real-world settings
- Finally, we surveyed the literature on different benchmarks used by researchers to assess performance of CUAs

Homework

- Read this presentation and focus on the paper from Wang, Xinyuan, et al. "Opencua: Open foundations for computer-use agents." *arXiv preprint arXiv:2508.09123* (2025).
 - Most importantly Section 1, 2, 3
- Contrast and compare the last two lectures to understand strengths and weaknesses of the two approaches